时钟之迹

STM32F407 时钟配置与软件延时

STM32F407ZGT6
Keil MDK / STM32CubeMX

168 MHz

1 ms

HAL 时基周期 · 由 HAL_InitTick 配置

01 初始化

02 设置

03 获取

04 应用

代码追踪：CH5EX01HAL / CH5Ex01STD

四阶段：初始化、设置、获取、应用

01

初始化

复位与运行环境

`SystemInit()`
`HAL_Init()`

02

设置

主频与时基配置

`SystemClock_Config()`
`SysTick_Config()`

03

获取

频率与时间读取

`HAL_RCC_Get…Freq()`
`HAL_GetTick()`

04

应用

延时与任务时序

`Delay()`
`HAL_Delay()`

CH5EX01HAL 外部时钟配置 8 MHz　　**CH5Ex01STD** 外部时钟配置 25 MHz

两份实际例程，主频均配置为 168 MHz（板上晶振差异见第 4 页）；沿调用关系分析配置与时序。

源码：HAL main.c、hal_conf.h；STD Usermain.c、system_stm32f4xx.c、stm32f4xx.h

初始化：启动流程与时基依赖

复位 → `SystemInit()` → C 运行时初始化 → `main()`　　两份工程共同的入口

标准库 • main 之前

`SystemInit()`

→ `SetSysClock()`

→ `HSE + PLL + 总线分频`

进入 main 时，主时钟配置已经完成。

HAL • main 中

main() / HAL 调用顺序

```
HAL_Init();
SystemClock_Config();
MX_GPIO_Init();
```

HSI 16 MHz 建立时基 → PLL 168 MHz。

初始 1 ms 时基供 HSE 就绪检测、PLL 锁定等待中的超时判断使用。
切换至 168 MHz 后：更新 SystemCoreClock → 再次调用 HAL_InitTick()。

源码：两工程启动文件与 system_stm32f4xx.c；HAL main.c、hal_rcc.c

PLL：主频合成与输出约束

HSE		M		N		P		SYSCLK	
8	÷	8	×	336	÷	2	=	**168**	MHz

SystemClock_Config() / PLL 参数

```
RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
RCC_OscInitStruct.PLL.PLLM = 8;
RCC_OscInitStruct.PLL.PLLN = 336;
RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
RCC_OscInitStruct.PLL.PLLQ = 4;
```

STD：25 ÷ 25 × 336 ÷ 2 = 168 MHz，原例程 Q = 7。

板上晶振 8 MHz：实际 53.76 MHz，SystemCoreClock 仍为 168 MHz，M 需改 25→8。

PLLQ 输出频率

84 MHz > 75 MHz

Q = 4：336 ÷ 4 = 84 MHz

超过数据手册规定的输出上限。

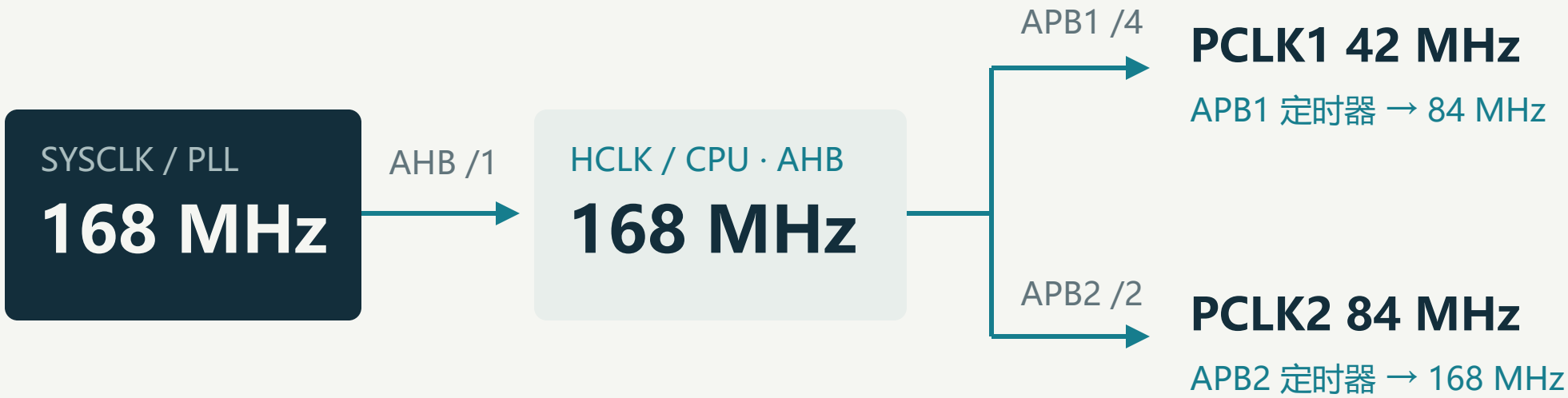
USB OTG FS、SDIO、RNG 使用 PLL48CK，例程未启用，越限不影响运行。

本组参数：Q = 7 → 48 MHz

DS8626 Rev 12，表 36：最大 75 MHz

源码：HAL main.c；STD system_stm32f4xx.c。数据手册：DS8626 Rev 12，p.106，表 36。

总线分频与 Flash 访问时序




```
RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;
```

168 MHz / 2.7–3.6 V：Flash 配置 5 WS（RM0090 Rev 22，表 11）。

F407 定时器时钟

APB 分频 = 1：TIM = PCLK

APB 分频 ≠ 1：TIM = 2 × PCLK

源码：HAL main.c；原理：RM0090，STM32F405/407 RCC 时钟树

获取：时钟频率与累计时间

读时钟频率 Hz

`HAL_RCC_GetSysClockFreq()`	168 000 000
`HAL_RCC_GetHCLKFreq()`	168 000 000
`HAL_RCC_GetPCLK1Freq()`	42 000 000
`HAL_RCC_GetPCLK2Freq()`	84 000 000

标准库在 Usermain.c 读取整组频率

```
RCC_GetClocksFreq(&RCC_Clocks);
```

读累计时间 ms

`HAL_GetTick()`

返回时基计数 uwTick。
默认每次中断累计 1 ms。

stm32f4xx_hal.c

```
__weak uint32_t HAL_GetTick(void)
{
  return uwTick;
}
```

GetSysClockFreq()：按寄存器与时钟源参数计算；GetHCLKFreq()：读取 SystemCoreClock。
SystemCoreClockUpdate() 刷新 HCLK 软件记录；HAL_GetTick() 读取中断维护的 uwTick。

源码：HAL hal_rcc.c 的 Get...Freq()；hal.c；STD Usermain.c

SysTick：时基周期与重装值

24 位递减计数器：装载 → 递减 → 到零中断 → 重装

`T = (LOAD + 1) / fSysTick`

两份例程均通过 CMSIS 选择 HCLK

标准库 · HCLK / 100

10 ms

ticks = 1 680 000　LOAD = 1 679 999

HAL 默认 · HCLK / 1000

1 ms

ticks = 168 000　LOAD = 167 999

```
SysTick_Config(RCC_Clocks.HCLK_Frequency / 100);
HAL_SYSTICK_Config(SystemCoreClock / (1000U / uwTickFreq))
```

原调用表达式；HAL 默认 uwTickFreq = 1。CMSIS 写入 LOAD = ticks − 1；改 HCLK 后需重新配置节拍。

源码：STD Usermain.c；HAL hal.c；CMSIS core_cm4.h:2022–2036；PM0214 SysTick

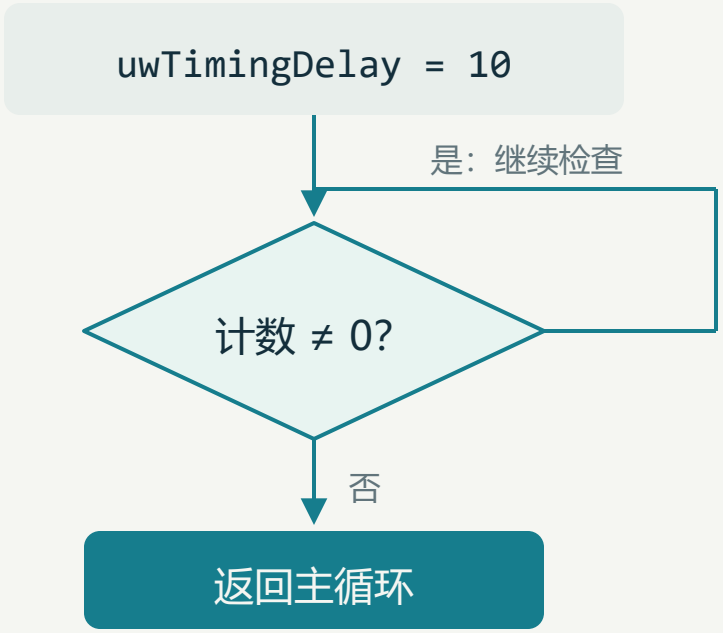
标准库 Delay：递减计数与阻塞等待

Usermain.c

```
void Delay(__IO uint32_t nTime)
{
  uwTimingDelay = nTime;
  while(uwTimingDelay != 0);
}

void TimingDelay_Decrement(void)
{
  if (uwTimingDelay != 0x00)
  {
    uwTimingDelay--;
  }
}
```

Delay(10) · 主循环



SysTick 每 10 ms 调用递减函数；Delay(10) ≈ 100 ms，Delay(1) ≈ 10 ms。板上 8 MHz 晶振时均为 3.1 倍。

__IO = volatile：保留共享计数的实际访问；循环阻塞当前任务，中断仍可执行。

源码：STD Usermain.c；stm32f4xx_it.c

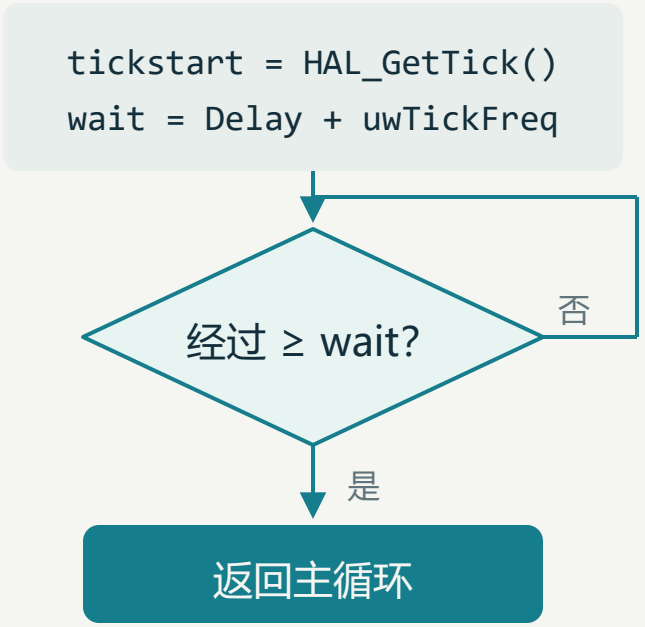
HAL_Delay：时间差判定与阻塞等待

stm32f4xx_hal.c

```
__weak void HAL_Delay(uint32_t Delay)
{
  uint32_t tickstart = HAL_GetTick();
  uint32_t wait = Delay;
  if (wait < HAL_MAX_DELAY)
  {
    wait += (uint32_t)(uwTickFreq);
  }
  while((HAL_GetTick() - tickstart) < wait)
  {
  }
}
```

wait 增加一个时基周期，补偿调用起点处于节拍末尾的情况。

HAL_Delay(1000) · 阻塞 1 s



节拍：SysTick 中断 → HAL_IncTick

参数单位 ms；节拍周期由 HAL_InitTick 配置。

源码：HAL main.c；stm32f4xx_hal.c；stm32f4xx_it.c

应用：LED 输出周期与计数回卷

标准库 • 模 5 条件触发

```
uint16_t Number=0,Index=0;
Delay(10);
if(Index%5==0)
  GPIO_ToggleBits(GPIOE, LED2_Pin);
Index++;
```

普通区间：翻转 ≈ 500 ms，完整亮灭周期 ≈ 1 s

流水灯每 100 ms 移位。

Index 回卷前后（每次循环约 100 ms）

65534 → 65535 → 0 → 1

条件不成立　翻转　再次翻转　条件不成立

HAL • 每轮翻转

```
HAL_Delay (1000);
HAL_GPIO_TogglePin(
  GPIOE, LED2_Pin);
```

翻转 ≈ 1 s，亮灭周期 ≈ 2 s

流水灯每 1 s 移位。

相邻两轮连续触发

500 ms → 100 ms

改用周期计数或时间差判断。

源码：CH5Ex01STD / Usermain.c；CH5EX01HAL / main.c

时钟域分工与外设门控

高速源与 PLL

HSI：内部 16 MHz RC，可用于启动与后备。
HSE：外部晶振，提供主时钟参考。
PLL：对参考时钟分频、倍频，再分配输出。

系统与总线时钟

SYSCLK：选定的系统时钟源输出。
HCLK：CPU、AHB 总线的节拍。
PCLK1 / PCLK2：APB 外设的节拍。

低速时钟

LSE：外部 32.768 kHz，适合 RTC 走时。
LSI：内部约 32 kHz，供独立看门狗等使用。

软件计时与硬件定时

SysTick：软件时基、超时判断、短时等待。
TIM / PWM：周期中断、精确定时与波形输出。

```
__HAL_RCC_GPIOE_CLK_ENABLE();
```

使用外设前，先打开对应的时钟门控。

gpio.c · 未使用的外设可关闭时钟，以降低功耗。

依据：RM0090，STM32F405/407 RCC、RTC、TIM；PM0214 SysTick；HAL gpio.c

时钟配置与软件时序的完整路径

时钟源、分频与计数共同决定软件时序。

初始化

启动路径

SystemInit / HAL_Init

设置

主频与时基配置

PLL / 总线 / SysTick

获取

频率与时间读取

Get...Freq / GetTick

应用

输出时序

Delay / HAL_Delay

标准库节拍 **10 ms** · HAL 节拍 **1 ms** · 两种延时均为阻塞等待

代码：教师例程 CH5EX01HAL、CH5Ex01STD（主程序、RCC/HAL驱动、中断与CMSIS文件）。

手册：ST RM0090（RCC / Flash）；PM0214（SysTick）；DS8626 Rev 12（PLL 输出指标）。